

Project Interim Report
On
Game Scoreboard System
(Using Core Java & OOP Concepts)

RU-100-01-00012
Object Oriented Programming With Java

SESSION: 2025-26



Submitted By
Pushkar Sahu
ERP - 11065

**Bachelor of Technology in Computer Science &
Engineering with AI and ML in association with
IBM**

Under the Guidance of
Mr. Ashish Shivhare

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(April 2026)

Abstract

The **Game Scoreboard System** is a console-based application developed using Core Java and Object-Oriented Programming (OOP) concepts. The main purpose of this project is to manage player information and maintain a leaderboard based on their scores. In many games, keeping track of scores and rankings is essential to evaluate player performance, and this system simulates that real-world functionality.

This project is needed to reduce manual effort in maintaining player scores and to ensure accurate and fast ranking of players. It provides an organized way to store player details such as name and score, and automatically sorts them to display rankings.

The system allows users to add multiple players, store their scores, sort them in descending order, and display a leaderboard with ranks. It also identifies the top player with the highest score. The project demonstrates the use of important Java concepts such as classes, objects, arrays, encapsulation, and sorting techniques, making it useful for beginners to understand practical implementation of OOP concepts.

Introduction

The **Game Scoreboard System** is a simple console-based application developed using Java programming language. It is designed to manage player details and maintain a leaderboard based on their scores. In many real-world games, scoreboards are used to rank players according to their performance. This project replicates that concept in a simplified way.

The system allows users to add player information such as name and score, store multiple player records, and sort them in descending order to generate rankings. It helps in reducing manual effort and errors that may occur while maintaining records on paper or spreadsheets.

This project also focuses on implementing key Object-Oriented Programming (OOP) concepts like classes, objects, encapsulation, and arrays of objects. It gives students practical exposure to how data can be organized and processed efficiently in Java.

Objectives of the Project

- Manage player records efficiently
- Store and update player scores
- Sort players based on performance
- Display ranked leaderboard
- Identify top scorer
- Apply OOP concepts in real-world scenario

Scope

The **Game Scoreboard System** is designed for basic score management and ranking of players in small-scale games or academic projects. It is suitable for scenarios where a simple and efficient system is required to store player details and display their performance in the form of a leaderboard.

This project can be used in schools, colleges, or small gaming events where maintaining player scores manually can be time-consuming and error-prone. It helps automate the process of ranking players based on their scores, ensuring accuracy and saving time.

Although the current system is console-based and uses arrays for data storage, it has the potential to be enhanced in the future. The project can be extended by integrating a database to store large amounts of data permanently. Additional features like a graphical user interface (GUI), online score tracking, multiplayer support, and advanced search or filtering options can also be added.

Thus, the project provides a strong foundation for building more advanced game management systems in the future.

System Requirements

To develop and run the **Game Scoreboard System**, the following hardware and software requirements are needed:

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Processor: Intel i3 or above (or equivalent)
- Keyboard and Monitor

Software Requirements

- Java Development Kit (JDK)
- IDE (Integrated Development Environment): VS Code / Eclipse / IntelliJ IDEA
- Operating System: Windows / Linux / macOS

These requirements are minimal and easily available, making the project simple to implement on most systems. The use of Java ensures platform independence, meaning the program can run on any system that supports Java.

Methodology

The **Game Scoreboard System** follows a simple step-by-step approach to manage player data and display rankings. The working of the system is explained below:

1. Start the Program

The program begins execution from the main method.

2. Add Player Details

The user enters player information such as name and score. These details are stored using objects of the Player class.

3. Store Data in Array

All player objects are stored in an array. This allows handling multiple players efficiently.

4. Process Data (Sorting)

The system sorts the players based on their scores in descending order (highest score first). A sorting technique like Bubble Sort or built-in

sorting can be used.

5. Display Leaderboard

After sorting, the system displays the leaderboard showing rank, player name, and score.

6. Show Top Player

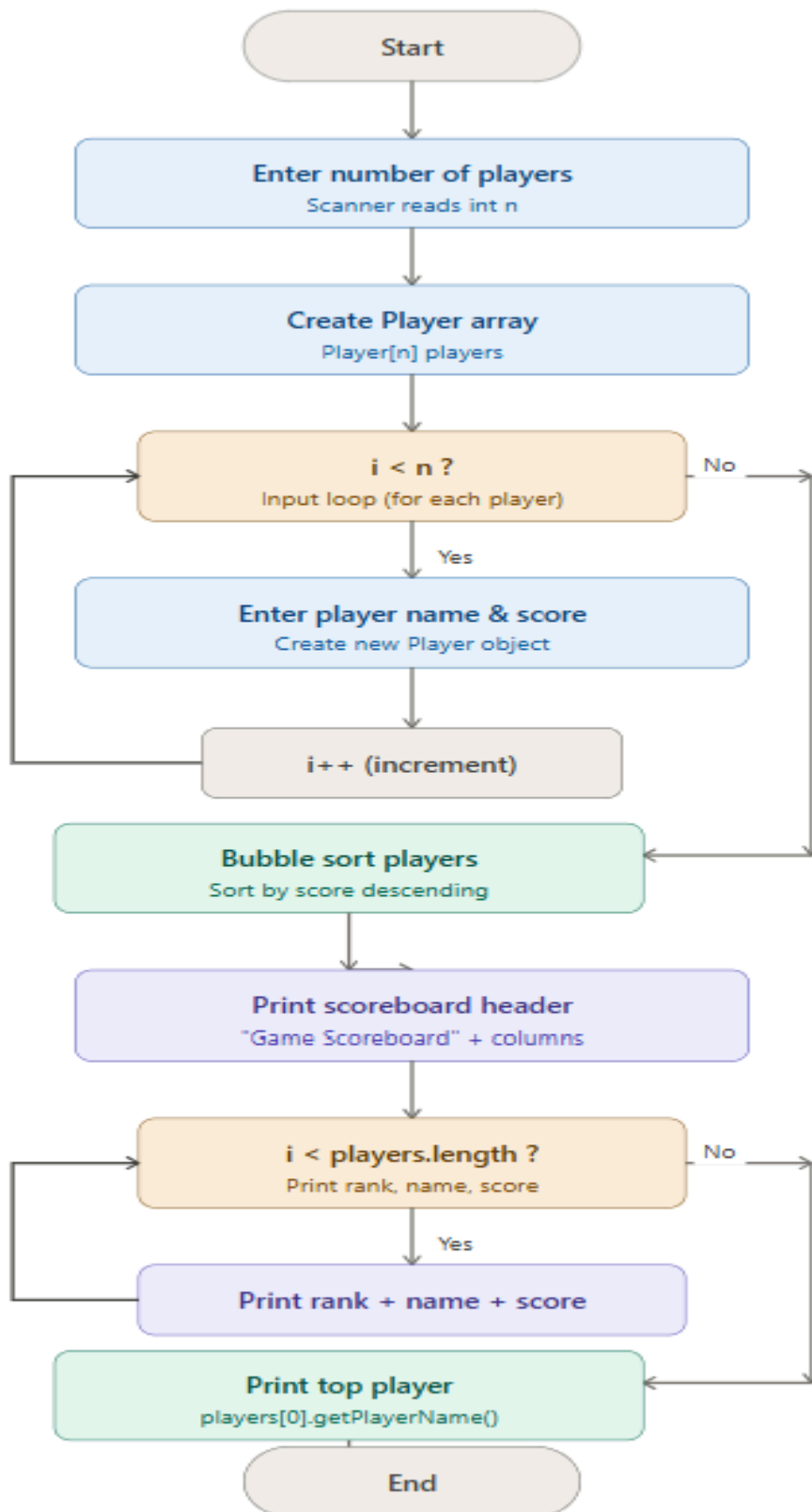
The player with the highest score is displayed separately as the top performer.

7. End Program

The program finishes after displaying all required results.

This step-by-step process ensures smooth execution and demonstrates how data is collected, processed, and displayed using Java and OOP concepts.

Flowchart & Class Diagram



Implementation (Explanation of Classes)

In this project, two main classes are used to implement the system:

1. Player Class

The **Player** class is used to store and manage individual player details.

- It contains two private variables:
 - **playerName** (String) → stores the name of the player
 - **score** (int) → stores the player's score
- A **constructor** is used to initialize these values when an object is created.
- **Getter methods** (**getName()**, **getScore()**) are used to access the values.
- **Setter methods** (**setName()**, **setScore()**) are used to modify the values if needed.

👉 This class follows **encapsulation**, as data is kept private and accessed through methods.

2. GameSystem Class

The **GameSystem** class controls the complete working of the program.

- It contains an **array of Player objects** to store multiple players:
 - **Player[] players**
- The **main() method** starts the program execution.
- It takes input from the user (player name and score).
- The program stores all player objects in the array.
- A **sorting method** is implemented to arrange players in descending order of scores.
- A **display method** prints the leaderboard with:
 - Rank
 - Player Name
 - Score
- It also displays the **top player** (highest score).

Sample Input / Output

Sample Input

Enter number of players: 4

Enter name of player 1: Rahul

Enter score of player 1: 95

Enter name of player 2: Ankit

Enter score of player 2: 88

Enter name of player 3: Priya

Enter score of player 3: 82

Enter name of player 4: Neha

Enter score of player 4: 76

Sample Output

Game Scoreboard

Rank	Player Name	Score
1	Rahul	95
2	Ankit	88
3	Priya	82
4	Neha	76

Top Player: Rahul

Future Enhancements

The current **Game Scoreboard System** is a basic console-based application. However, it can be improved and extended with several advanced features in the future:

- **User-Friendly Interface (GUI)**
Replace the console-based system with a graphical user interface using Java Swing or JavaFX for better user experience.
- **Database Integration**
Store player data in a database (like MySQL) instead of arrays to handle large amounts of data and ensure permanent storage.
- **Dynamic Data Handling**
Use `ArrayList` instead of arrays to allow flexible number of players.
- **Search Functionality**
Add a feature to search for a specific player by name.
- **Update/Delete Records**
Allow modification or deletion of player details.
- **Menu-Driven Program**
Provide options like Add Player, View Leaderboard, Search Player, Exit, etc.
- **Online Multiplayer Support**
Extend the system to support online players and real-time score updates.
- **Sorting Options**
Allow sorting based on different criteria like name or score.

These enhancements can make the system more powerful, scalable, and suitable for real-world applications.

Gantt Chart

Conclusion

The **Game Scoreboard System** project successfully demonstrates the use of Core Java and Object-Oriented Programming (OOP) concepts in a practical application. The system efficiently manages player details, stores scores, and generates a ranked leaderboard based on performance.

Through this project, important programming concepts such as classes, objects, encapsulation, arrays of objects, and sorting techniques have been implemented and understood clearly. The project also highlights how real-world systems, like game scoreboards, can be simplified and developed using basic programming logic.

The application reduces manual effort, ensures accurate ranking, and provides a structured way to handle player data. Although it is a simple console-based system, it serves as a strong foundation for developing more advanced applications in the future.

Overall, the project meets its objectives and provides a clear understanding of how Java can be used to build efficient and organized systems.

References

Books

[1] H. Schildt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill, 2018.

[2] E. Balagurusamy, *Programming with Java*, 5th ed. New Delhi: McGraw-Hill Education, 2014.

Websites

[3] Oracle, "Java Documentation," Oracle. [Online]. Available: <https://docs.oracle.com>

[4] GeeksforGeeks, "Java Programming Language," GeeksforGeeks. [Online]. Available: <https://www.geeksforgeeks.org>